ФЕДЕРАЛЬОНЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗАВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ “НАЦИОНАЛЬНЫЙ ИСЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет Программной Инженерии и Компьютерной Техники
Дисциплина: «Программирование»

ОТЧЁТ

по лабораторной работе №3
Вариант №66666

Выполнил:

Студент группы P3116

Сивков Александр Владимирович

Проверил:
Преподаватель
Шнейдерис Герардас Герардович

Санкт-Петербург 2025 г.

Содержание

Текст задания 3
Исходный код программы 6
Диаграмма классов объектной модели 7
Анализ кода, сгенерированного ИИ-ассистентами 8
Результат работы программы 16
Выводы по работе 17

Текст задания

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Описание предметной области, по которой должна быть построена объектная модель:

После всего предыдущего мы не очень удивились находке, скажу больше -- были почти к ней готовы. Однако когда, склонившись над санями, развязали брезентовый тюк, очертания которого меня почему-то смутно встревожили, нас как громом поразило. По-видимому, существам, побывавшим в лагере, тоже не была чужда страсть к научной систематизации, как и Лейку: в санях лежали два свежезамороженных экземпляра, раны вокруг шеи аккуратно залеплены пластырем, а дабы избежать дальнейших повреждений, сами тела туго перевязаны. Надо ли говорить, что то были Гедни и пропавшая собака. Наверное, многие сочтут нас бездушными и, конечно же, не вполне нормальными, но и после этого жуткого открытия мы продолжали думать о северном туннеле, хотя, уверяю, мысль о дальнейшем путешествии на какое-то время оставила нас, вытесненная другими размышлениями. Закрыв тело Гедни брезентом, мы стояли над ним в глубокой задумчивости, из которой нас вывели непонятные звуки -- первые, услышанные с того момента, как мы покинули улицы города, где слабо шелестел ветер, спускаясь со своих заоблачных высот. Очень земные и хорошо знакомые нам звуки были настолько неожиданны в этом мире пагубы и смерти, что, опрокидывая все наши представления о космической гармонии, ошеломили нас сильнее, чем это сделали бы самые невероятные звучания и шумы.

Этапы выполнения работы:

1. Получить вариант
2. Нарисовать UML-диаграмму, представляющую классы и интерфейсы объектной модели и их взаимосвязи;
3. Придумать сценарий, содержащий действия персонажей, аналогичные приведенным в исходном тексте;
4. <u>Согласовать диаграмму классов и сценарий с преподавателем.</u>

5. Написать программу на языке Java, реализующую разработанные объектную модель и сценарий взаимодействия и изменения состояния объектов. При запуске программа должна проигрывать сценарий и выводить в стандартный вывод текст, отражающий изменение состояния объектов, приблизительно напоминающий исходный текст полученного отрывка.
6. Для одного из методов класса персонажа (или объекта), согласованного с преподавателем, создать промпты для 2-3 ИИ-ассистентов (Gigachat, DeepSeek, Qwen, или др.), получить от них код метода, **реализовать метод самостоятельно**, и включить в отчет сравнительный анализ кода, написанного ИИ-ассистентами, и своего варианта. Оценить качество кода, полученного от ИИ-ассистентов, указать на возможные недоработки.
7. Продемонстрировать выполнение программы на сервере helios.
8. Ответить на контрольные вопросы и выполнить дополнительное задание.

Текст, выводящийся в результате выполнения программы не обязан дословно повторять текст, полученный в исходном задании. Также не обязательно реализовывать грамматическое согласование форм и падежей слов выводимого текста.

Стоит отметить, что цель разработки объектной модели состоит не в выводе текста, а в эмуляции объектов предметной области, а именно их состояния (поля) и поведения (методы). Методы в разработанных классах должны изменять состояние объектов, а выводимый текст должен являться побочным эффектом, отражающим эти изменения.

Требования к объектной модели, сценарию и программе:

1. В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.
2. Объектная модель должна реализовывать основные принципе ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.
3. Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей

среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).

4. Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:
    - абстрактный класс как минимум с одним абстрактным методом;
    - интерфейс;
    - перечисление (enum);
    - запись (record);
    - массив или ArrayList для хранения однотипных объектов;
    - проверяемое исключение.
5. В созданных классах основных персонажей и предметов должны быть корректно переопределены методы equals(), hashCode() и toString(). Для классов-исключений необходимо переопределить метод getMessage().
6. Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно unchecked исключение (можно свое, можно из стандартной библиотеки).
7. При необходимости можно добавить внутренние, локальные и анонимные классы.

Исходный код программы

Размещён на GitHub
https://github.com/Fragtoor/ProgLab3

Диаграмма классов объектной модели

Размещена на сайте:
https://drive.google.com/file/d/1GvzwVfWb5enzPmJsrZsvBg8ikgnNaIG0/view?usp=sharing

Анализ кода, сгенерированного ИИ-ассистентами

Выбранный метод для анализа:
Метод hearSound(SoundType sound) класса Character, который обрабатывает реакцию персонажа на различные звуки.

Промпты для ИИ-ассистентов

- Промпт для DeepSeek:
Напиши метод hearSound для класса Character на Java. Метод принимает параметр SoundType sound (enum).
В зависимости от типа звука метод должен:
1. Менять состояние персонажа (CharacterState) через changeState()
2. Выводить в консоль нарративные сообщения, соответствующие звуку
3. Использовать switch expression (Java 17+)
4. Обрабатывать все варианты SoundType enum
5. Учитывать, что персонаж может быть заморожен - в этом случае он не должен реагировать

SoundType включает: WIND, FAMILIAR_EARTHLY, INCREDIBLE_NOISE, SILENCE, SCREAM, UNSETTLING_WHISPER, BARKING, LAUGHTER
CharacterState включает: NORMAL, SHOCKED, THOUGHTFUL, DETERMINED, DEATH, PANIC, PARANOID

- Промпт для ChatGPT (GPT-4)
Реализуй надежный метод hearSound на Java, который:
1. Принимает параметр SoundType enum
2. Изменяет эмоциональное состояние персонажа на основе звука
3. Печатает захватывающие повествовательные сообщения
4. Включает защитное программирование (проверки на отсутствие данных, проверка замороженного состояния)
5. Использует улучшенный переключатель со стрелками синтаксиса
6. Обрабатывает нестандартные ситуации (неожиданные звуки, зависшие символы)
7. Возвращает логическое значение, указывающее, был ли звук успешно обработан

8. Следует принципам чистого кода с осмысленными именами переменных

Код, сгенерированный ИИ-ассистентами

- DeepSeek

```
/**
 * Реагирует на услышанный звук
 * @param sound тип звука
 */
public void hearSound(SoundType sound) {
    // Проверка на замороженное состояние
    if (this.isFrozen) {
        System.out.println(getName() + " заморожен и не может слышать звуки.");
        return;
    }

    // Проверка на null
    if (sound == null) {
        System.out.println(getName() + " услышал неопределенный звук.");
        changeState(CharacterState.PARANOID);
        return;
    }

    // Обработка звуков с помощью switch expression
    String reaction = switch (sound) {
        case WIND -> {
            changeState(CharacterState.THOUGHTFUL);
            yield "Просто ветер... мы почти успокоились.";
        }
        case FAMILIAR_EARTHLY -> {
```

```
            changeState(CharacterState.SHOCKED);
            yield "Хм, очень земные и хорошо знакомые мне звуки. Я очень поражён...";
        }
        case INCREDIBLE_NOISE -> {
            changeState(CharacterState.PARANOID);
            yield "Невероятные шумы... но мы были к ним готовы.";
        }
        case SILENCE -> {
            changeState(CharacterState.NORMAL);
            yield "Эта тишина меня успокаивает.";
        }
        case SCREAM -> {
            changeState(CharacterState.PANIC);
            yield "Крик! Что происходит?";
        }
        case UNSETTLING_WHISPER -> {
            changeState(CharacterState.SHOCKED);
            yield "Тревожный шепот леденит душу...";
        }
        case BARKING -> {
            changeState(CharacterState.THOUGHTFUL);
            yield "Собачий лай... может, Джэк?";
        }
        case LAUGHTER -> {
            changeState(CharacterState.NORMAL);
            yield "Чей-то смех... становится немного спокойнее.";
        }
    };

    System.out.println(getName() + ": \"" + reaction + "\"");
}
```

- ChatGPT (GPT-4)

```
/**
 * Processes an auditory stimulus and updates character state accordingly.
 *
 * @param sound The type of sound heard
 * @return true if sound was successfully processed, false if character couldn't react
 * @throws IllegalArgumentException if sound is null
 */
public boolean hearSound(SoundType sound) {
    // Defensive programming: validate input
    if (sound == null) {
        throw new IllegalArgumentException("Sound cannot be null");
    }

    // Check character condition
    if (isFrozen || state == CharacterState.DEATH) {
        System.out.println("[DEBUG] " + getName() + " cannot react to sound due to condition.");
        return false;
    }

    // Store previous state for comparison
    CharacterState previousState = this.state;
    String narrativeMessage = "";

    // Process sound using enhanced switch
    switch (sound) {
        case WIND -> {
            this.state = CharacterState.THOUGHTFUL;
            narrativeMessage = "Просто ветер... мы почти успокоились.";
        }
        case FAMILIAR_EARTHLY -> {
```

```
            this.state = CharacterState.SHOCKED;
            narrativeMessage = "Хм, очень земные и хорошо знакомые мне звуки. Я очень поражён...";
        }
        case INCREDIBLE_NOISE -> {
            this.state = CharacterState.PARANOID;
            narrativeMessage = "Невероятные шумы... но мы были к ним готовы.";
            // Special case: might trigger cosmic dissonance
            if (Math.random() > 0.7) {
                throw new CosmicDissonanceException("Sound triggered cosmic harmony disruption");
            }
        }
        case SILENCE -> {
            this.state = CharacterState.NORMAL;
            narrativeMessage = "Эта тишина меня успокаивает.";
        }
        case SCREAM -> {
            this.state = CharacterState.PANIC;
            narrativeMessage = "Крик! Что происходит?";
        }
        case UNSETTLING_WHISPER -> {
            this.state = CharacterState.SHOCKED;
            narrativeMessage = "Тревожный шепот леденит душу...";
        }
        case BARKING -> {
            this.state = CharacterState.THOUGHTFUL;
            narrativeMessage = "Собачий лай... может, Джэк?";
        }
        case LAUGHTER -> {
            this.state = CharacterState.NORMAL;
            narrativeMessage = "Чей-то смех... становится немного спокойнее.";
```

```
        }
        default -> {
            // Handle unexpected enum values
            this.state = CharacterState.PARANOID;
            narrativeMessage = "Неопознанный звук... это меня тревожит.";
        }
    }

    // Output narrative if state changed
    if (!previousState.equals(this.state)) {
        System.out.println(getName() + " (состояние: " + previousState + " → " + this.state + "): "
+ narrativeMessage);
    } else {
        System.out.println(getName() + ": " + narrativeMessage);
    }

    // Log the event
    logSoundEvent(sound, previousState, this.state);

    return true;
}

private void logSoundEvent(SoundType sound, CharacterState oldState, CharacterState
newState) {
    // In production, this would write to a log file
    System.out.println("[LOG] " + getName() + " heard " + sound +
                    " | State change: " + oldState + " -> " + newState);
}
```

Мой вариант:

```
public void hearSound(SoundType sound) {
    String result = "";
```

```
    switch(sound) {
        case FAMILIAR_EARTHLY:
            result += "\033[3m - Хм, очень земные и хорошо знакомые мне звуки. Я очень поражён...\033[0m";
            changeState(CharacterState.SHOCKED);
            break;
        case INCREDIBLE_NOISE:
            result += "\033[3m - Невероятные шумы... но мы были к ним готовы.\033[0m";
            changeState(CharacterState.PARANOID);
            break;
        case UNSETTLING_WHISPER:
            result += "\033[3m - Тревожный шепот леденит душу...\033[0m";
            changeState(CharacterState.SHOCKED);
            break;
        case WIND:
            result += "\033[3m - Просто ветер... мы почти успокоились.\033[0m";
            changeState(CharacterState.THOUGHTFUL);
            break;
        case SILENCE:
            result += "\033[3m - Эта тишина меня успокаивает.\033[0m";
            changeState(CharacterState.NORMAL);
            break;
        case SCREAM:
            result += "\033[3m - Крик! Что происходит?\033[0m";
            changeState(CharacterState.PANIC);
            break;
        case BARKING:
            result += "\033[3m - Собачий лай... может, Джэк?\033[0m";
            changeState(CharacterState.THOUGHTFUL);
            break;
    }
```

```
    System.out.println(result);
}
```

Мой вариант наиболее лаконичен (30 строк) и идеально интегрирован в проект, но отсутствует проверка ошибок. DeepSeek предлагает баланс (50 строк) с базовой валидацией и switch expression, однако нарушает инкапсуляцию прямым доступом к полям. ChatGPT-версия (80 строк) наиболее профессиональна, но слишком перегружена для учебной задачи. Все три варианта корректно реализуют основную логику реакции на звуки, но отличаются уровнем безопасности и сложности. Для данного проекта, по моему мнению, оптимален мой вариант.

Результат работы программы

Сценарий, напоминающий предложенный отрывок из произведения. При каждом запуске его ход может меняться, так как предусмотрена вариативность

```
   После всего предыдущего исследователи не очень удивились находке,
скажу больше -- они были почти к ней готовы. Однако когда, склонившись над санями,
развязали брезентовый тюк, они обнаружили нечто неожиданное...

 - В санях два свежезамороженных экземпляра... Гедни и пропавшая собака Джэк, видимо, породы Хаски - сказал Лейк.

   После увиденного:
- Лейк шокирован находкой.
- Кейси сохраняет хладнокровие.

   Немного придя в себя после этого неприятного зрелища,
исследователи начали размышлять о туннеле:
Лейк думает: несмотря ни на что, нужно добраться до северного туннеля...
Кейси думает: несмотря ни на что, нужно добраться до северного туннеля...

   Внезапно появляется звук ветра, и исследователи говорят:
 - Просто ветер... мы почти успокоились.
```

Пример

Выводы по работе

Чему я научился, выполнив лабораторную работу:

В ходе выполнения лабораторной работы была успешно разработана объектно-ориентированная модель на основе литературного отрывка. Основной целью - создание эмуляции объектов предметной области с их состояниями и поведением - достигнута через реализацию иерархии классов, интерфейсов и взаимодействий между объектами.